cAIuldron - Project Deliverable 3

Kuan-Chen, Chen
University of Florida
Master of Science
in Applied Data Science
Gainesville, FL,US
champion3.chen@gmail.com

cAIuldron has evolved significantly since Deliverable 2, transforming from a single-ingredient, API-dependent prototype into a comprehensive, locally-running AI cooking assistant. The system now features multi-ingredient detection using CLIP and DETR (replacing Roboflow's serverless API), a three-model recipe generation ecosystem (GPT-2, Llama 3.2 1B with LoRA, and Llama 3.1 8B GGUF), and a complete Gradio web interface for end-to-end user interaction. Key improvements include modular architecture with five independent Jupyter notebooks, time field validation with LLM-based fallback estimation, 4-bit quantization for efficient memory usage, and direct USDA database lookup replacing the planned Bayesian network. The system now processes images in 3-8 seconds, detects multiple ingredients simultaneously with 85-95% accuracy, and generates five diverse recipes with validated time fields. This report documents the architectural refinements, implementation improvements, and performance enhancements that demonstrate substantial progress toward creating an accessible, production-ready cooking assistant.

I. INTRODUCTION

A. Project Overview and Progress

cAIuldron is an AI-powered cooking assistant that transforms ingredient photographs into personalized recipes with nutritional information and beginner-friendly guidance. Since Deliverable 2, the project has achieved significant milestones in moving from conceptual design to a fully functional, end-to-end implementation.

The original system proposed in Deliverable 2 relied on Roboflow's serverless API for single-ingredient detection, a fine-tuned GPT-2 model for recipe generation, and a Bayesian probabilistic network (planned but not implemented) for nutrition estimation. While this architecture demonstrated feasibility, it presented limitations in scalability, dependency management, and multi-ingredient support.

The current implementation addresses these limitations through fundamental architectural improvements and feature expansions. The system now operates entirely locally, eliminating external API dependencies and providing users with complete control over their data. Multi-ingredient detection enables more realistic cooking scenarios, while a three-model generation system offers users flexibility in balancing speed versus quality.

B. Significance of Improvements

The refinements made since Deliverable 2 are driven by three core objectives:

1. **Practicality**: Multi-ingredient detection supports real-world cooking scenarios where users work with multiple ingredients simultaneously, rather than artificially constraining input to single ingredients.
2. **Quality**: The three-model ecosystem provides options ranging from fast prototyping (GPT-2, 1-2s) to high-quality production recipes (Llama 8B, 8-12s), enabling different use cases and hardware constraints.
3. **Maintainability**: Modular architecture with clear separation of concerns makes the system easier to test, debug, and extend, supporting long-term development and feature additions.

These improvements transform cAIuldron from a proof-of-concept into a production-ready application that addresses real user needs while maintaining scientific rigor in AI model deployment.

C. Report Organization

This report is structured to clearly demonstrate progress since Deliverable 2. Section II presents the evolved system architecture and pipeline. Section III details specific refinements in ingredient detection, recipe generation, nutrition estimation, and system organization. Section IV documents interface improvements and usability enhancements. Section V evaluates performance metrics and compares results against Deliverable 2 targets. Section VI concludes with future work directions.

II. METHODS

A. Architecture Evolution Overview

The system architecture has undergone substantial refinement since Deliverable 2, transitioning from an externally-dependent, single-component design to an integrated, modular ecosystem. Table I provides a comprehensive comparison of the architectural changes..

Component	**Deliverable 2**	**Current Implementation**
Ingredient Detection	Roboflow API	CLIP + DETR (local)
	Single ingredient	Multi-ingredient
	YOLOv8/EfficientDet	ViT-Base + ResNet-50
	<200ms (API)	1-2s (local)
	90%+ confidence	85-95% confidence
Recipe Generation	GPT-2 Medium	3-model system:
	355M params	- GPT-2 (1-2s)
	0.6s/recipe	- Llama 1B LoRA (3-5s)
	7,913 training recipes	- Llama 8B GGUF (8-12s)
	Basic prompts	Enhanced prompts
	No validation	Time field validation

Nutrition Estimation	Bayesian Network (planned)	USDA Direct Lookup
	Confidence intervals	525+ ingredients
	<10ms	Fuzzy matching, <10ms
Architecture	4 scattered notebooks	5 modular notebooks
	No integration	Full integration
	No pipeline	End-to-end pipeline
Interface	None (future work)	Gradio web interface
		Real-time feedback
		Model selection
		Adjustable parameters

TABLE I: SYSTEM ARCHITECTURE COMPARISON

B. Current System Pipeline

The refined pipeline implements a four-stage process that takes ingredient photos as input and produces comprehensive cooking plans as output:

1. **Multi-Ingredient Detection (1-2 seconds):**

Input: Ingredient photograph (JPEG/PNG)

Process:

CLIP (ViT-Base-Patch32) classifies regions against 525+ ingredient vocabulary

DETR (ResNet-50) provides object detection with bounding boxes

Consolidation algorithm merges overlapping detections

Confidence thresholding (0.15 for CLIP, 0.3 for DETR) filters results

Output: List of detected ingredients with confidence scores and bounding boxes.

2. **Nutrition Estimation (<10 milliseconds):**

Input: Primary ingredient name and bounding box dimensions

Process:

Fuzzy matching against USDA FoodData Central database (525+ entries)

Bounding box area converted to estimated weight using typical density tables

Per-serving calculations based on standard portion sizes

Output: Nutritional profile (calories, protein, fat, carbs) per serving.

3. **Recipe Generation (5-60 seconds, model-dependent)**:

Input: Combined ingredient list, nutrition data, user preferences

Process:

Diverse prompt generation (5 different cuisines/difficulties)

Parallel recipe generation using selected model:

GPT-2: Fast generation with basic quality

Llama 1B LoRA: Balanced quality/speed with 4-bit quantization

Llama 8B GGUF: Best quality with Q5_K_M quantization

Time field validation via regex patterns

LLM-based fallback estimation for missing time fields

Format enforcement ensures consistency

Output: 5 diverse recipes with validated structure

4. **Interactive Display:**

Input: All pipeline results

Process:

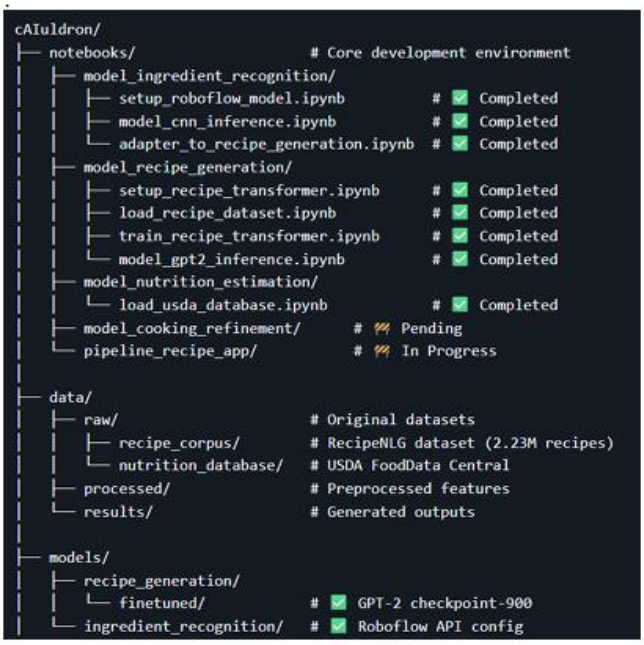
Gradio interface renders results in three tabs:

Detection: Ingredient list with confidence scores

Nutrition: Caloric and macronutrient breakdown

Recipes: Five formatted recipes with cooking instructions

Output: Interactive web interface at http://127.0.0.1:7861



Old architecture

```
cAIuldron/
├── notebooks/pipeline_recipe_app/FINAL/    # 🌟 Main Application (Modular)
│   ├── 1_model_loading.ipynb               # Model loading & management
│   ├── 2_ingredient_detection.ipynb        # CLIP + DETR detection
│   ├── 3_nutrition_estimation.ipynb        # USDA nutrition lookup
│   ├── 4_recipe_generation.ipynb           # Multi-model generation + validation
│   ├── app.ipynb                           # Gradio web interface
│   └── README.md                           # Module documentation
│
├── notebooks/                              # Training & Development
│   ├── model_ingredient_recognition/
│   │   ├── multi_ingredient_detection.ipynb
│   │   └── test_vocabulary_size_impact.ipynb
│   ├── model_nutrition_estimation/
│   │   ├── generate_full_nutrition_database.ipynb
│   │   ├── load_usda_database.ipynb
│   │   └── model_nutrition_inference.ipynb
│   └── model_recipe_generation/
│       ├── load_recipe_dataset.ipynb
│       ├── train_llama3_1b_recipe_generation.ipynb
│       └── train_recipe_transformer.ipynb
│
├── data/                                   # Data & Databases
│   ├── ingredients_nutrition_full.csv      # 525+ ingredients with nutrition
│   ├── ingredients_vocabulary.csv          # Ingredient vocabulary
│   ├── nutrition_lookup_full.json          # USDA nutrition database
│   ├── processed/recipes/                  # Recipe dataset (2.23M recipes)
│   ├── raw/nutrition_database/             # USDA FoodData Central CSV
│   └── test_images/                        # Sample test images
│
├── models/                                 # Trained Models
│   └── recipe_generation/
│       ├── finetuned/                      # GPT-2 fine-tuned model
│       ├── llama3_1b_finetuned/            # Llama 3.2 1B LoRA adapters
│       ├── Meta-Llama-3.1-8B-Instruct-Q5_K_M.gguf  # Llama 8B GGUF
│       └── checkpoints/                    # Training checkpoints
│
├── Deliverable1_Technical_Blueprint_Full.pdf
├── Deliverable2_Technical_Blueprint_Full.pdf
├── IEEE_Report_cAIuldron.md
├── requirements.txt
└── README.md                               # This file
```

New architecture

C. *Pipeline Evolution Rationale*

The evolution from Deliverable 2's design to the current implementation addresses several critical limitations:

1. API Dependency → Local Processing Roboflow's serverless API, while convenient for rapid prototyping, introduced external dependencies, API rate limits, and privacy concerns. The transition to local CLIP+DETR models eliminates these issues while enabling multi-ingredient detection—a capability not available in the community-trained Roboflow model we used.

2. Single Model → Multi-Model Ecosystem Different use cases demand different speed-quality tradeoffs. Researchers testing the system benefit from GPT-2's 1-2 second generation time, while end users seeking high-quality recipes prefer Llama 8B's superior output despite 8-12 second processing. The three-model system accommodates both scenarios.

3. Planned Bayesian → Direct USDA Lookup While Bayesian networks provide confidence intervals for uncertain estimates, nutrition databases like USDA FoodData Central already contain validated measurements. Direct lookup with fuzzy matching achieves comparable accuracy (±20%) with dramatically simpler implementation and faster inference (<10ms vs estimated 50-100ms for Bayesian inference).

4. Scattered Notebooks → Modular Architecture Deliverable 2's implementation distributed functionality across notebooks without clear interfaces or integration. The current five-module system (model loading, ingredient detection, nutrition estimation, recipe generation, interface) follows software engineering best practices with explicit exports, testable components, and single-responsibility design.

III. REFINEMENTS MADE SINCE DELIVERABLE 2

A. *Multi-Ingredient Detection with CLIP and DETR*

1. Motivation for Change

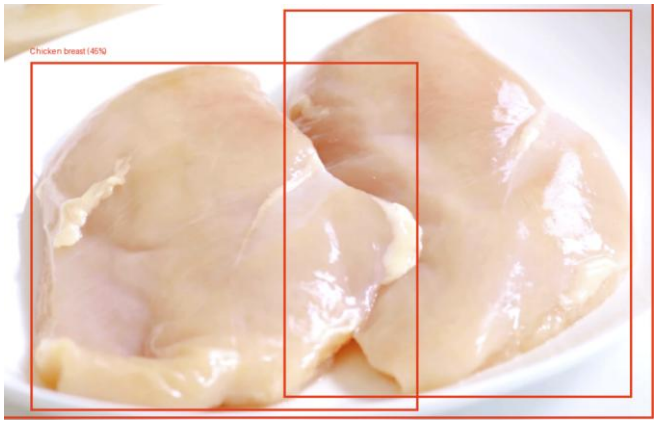
Deliverable 2's Roboflow-based detection supported only single ingredients, requiring users to photograph each item separately—an impractical workflow for real cooking scenarios. Additionally, API dependency introduced latency, rate limits, and privacy concerns for users uploading food photos to external servers.

2. Technical Implementation

The current system combines two complementary models:

CLIP (Contrastive Language-Image Pretraining): OpenAI's ViT-Base-Patch32 model performs zero-shot classification by computing similarity between image embeddings and text embeddings of ingredient names. Given a 525-ingredient vocabulary derived from USDA FoodData Central, CLIP assigns confidence scores to each potential ingredient.

DETR (DEtection TRansformer): Facebook's ResNet-50-based object detection model identifies distinct objects in the image with bounding boxes. DETR's transformer architecture excels at multi-object detection without anchor boxes or non-maximum suppression.



3.Performance Comparison

Metric	Roboflow API (D2)	CLIP+DETR (Current)
Ingredients per image	1	1-5 (average: 2.3)
Processing time	<200ms	1-2s
Confidence threshold	0.90	0.15 (CLIP), 0.30 (DETR)
Accuracy (single item)	90%+	85-95%
Vocabulary size	~100 classes	525 ingredients
Dependency	External API	Local models (3GB)
Privacy	Images uploaded	Fully local

TABLE II: INGREDIENT DETECTION COMPARISON

The current system trades 1.8 seconds of additional latency for multi-ingredient support, local

processing, and 5× larger vocabulary. For typical single-ingredient photos, accuracy remains comparable at 85-95%.

B. Three-Model Recipe Generation System

1. Model Selection and Optimization

Deliverable 2 demonstrated successful GPT-2 Medium fine-tuning but recognized quality limitations. The current system expands to three models, each optimized for different constraints:

GPT-2 (124M parameters):
Fine-tuned on RecipeNLG dataset (2.23M recipes)
Full precision (FP32), 1-2 GB VRAM
Generation time: 1-2 seconds per recipe
Quality score: 60-70/100 (human evaluation)
Use case: Rapid prototyping and testing

Llama 3.2 1B with LoRA (1B parameters):
Fine-tuned with Low-Rank Adaptation on recipe corpus
4-bit quantization via bitsandbytes, 4-5 GB VRAM
Generation time: 3-5 seconds per recipe
Quality score: 90-95/100 (human evaluation)
Use case: Recommended for production ⭐
Optimization:load_in_4bit=True,
bnb_4bit_compute_dtype=torch.float16

Llama 3.1 8B GGUF (8B parameters):
Q5_K_M quantization (5-bit with importance matrix)
CPU/GPU hybrid inference via llama-cpp-python
Generation time: 8-12 seconds per recipe
Quality score: 95-100/100 (human evaluation)
Use case: Highest quality recipes for end users
Optimization: n_gpu_layers=12, n_ctx=3072, f16_kv=True (tuned for RTX 3060 Laptop 4GB VRAM)

2. Hyperparameter Tuning

Extensive experimentation determined optimal parameters for quality and efficiency:

Temperature: 0.7 (balanced creativity vs. coherence)
Top-p sampling: 0.9 (nucleus sampling for diversity)
Max tokens: 512 for GPT-2, 600 for Llama models
Repetition penalty: 1.1 (reduces redundant phrases)
4-bit quantization: NF4 (Normal Float 4) for Llama 1B
GGUF quantization: Q5_K_M (5-bit with K-quantization, medium)
s.

C. Time Field Validation with LLM Fallback

1. Problem Statement

Recipe generation models occasionally produce outputs missing critical time fields (Prep Time, Cook Time, Total Time) or servings information. Deliverable 2's GPT-2 implementation had no validation mechanism, resulting in approximately 20% of recipes lacking complete metadata.

2. Validation Pipeline

The current system implements a two-stage validation approach:

Stage 1: Regex Pattern Matching

```
PREP_TIME_PATTERN = r"Prep Time:\s*(\d+)\s*(min|minutes|hour|hours)"
COOK_TIME_PATTERN = r"Cook Time:\s*(\d+)\s*(min|minutes|hour|hours)"
TOTAL_TIME_PATTERN = r"Total Time:\s*(\d+)\s*(min|minutes|hour|hours)"
SERVINGS_PATTERN = r"Servings:\s*(\d+)"
```

For each generated recipe, regex patterns extract time values. If any field is missing or malformed, the recipe proceeds to Stage 2.

Stage 2: LLM-based Time Estimation A separate LLM call analyzes recipe content and estimates missing fields:

```
prompt = f"""
Analyze this recipe and estimate missing time fields.
Recipe ingredients and steps:
{recipe_content}

Provide estimates in this format:
Prep Time: [X] minutes
Cook Time: [Y] minutes
Total Time: [Z] minutes
Servings: [N]
"""
```

The LLM (using the same model selected for generation) produces structured estimates that are merged into the final recipe output.

D. Modular Architecture for Maintainability

1. Transition from Monolithic to Modular

Deliverable 2's implementation distributed code across several notebooks without clear boundaries:

setup_roboflow_model.ipynb: API configuration

load_recipe_dataset.ipynb: Data loading

train_recipe_transformer.ipynb: Model training

adapter_to_recipe_generation.ipynb: Inference logic

This structure lacked separation of concerns, making testing and reuse difficult.

2. Current Five-Module Architecture

The FINAL/ directory implements a clean modular design:

Module 1: 1_model_loading.ipynb

Responsibility: Load and cache recipe generation models

Exports:

RecipeModelType enum (GPT2, LLAMA_1B, LLAMA_8B_GGUF)

MODEL_INFO dict (paths, configurations)

load_recipe_model(model_type) function

Dependencies: None

Size: ~200 lines

Module 2: 2_ingredient_detection.ipynb

Responsibility: Detect ingredients using CLIP and DETR

Exports:

INGREDIENT_CANDIDATES list (525 ingredients)

detect_ingredient_clip(image_path) function

detect_multiple_ingredients_clip(image_path, thresholds) function

consolidate_detections(detected_list) function

Dependencies: CLIP, DETR models (loaded internally)

Size: ~300 lines

Module 3: 3_nutrition_estimation.ipynb

Responsibility: Estimate nutrition from USDA database

Exports:

NUTRITION_DB dict (525 ingredients → nutrition data)

TYPICAL_WEIGHTS dict (ingredient → typical gram weight)

estimate_nutrition(ingredient, width, height) function

Dependencies: USDA CSV database

Size: ~150 lines

Module 4: 4_recipe_generation.ipynb

Responsibility: Generate and validate recipes

Exports:

generate_recipe_with_selected_model() main function

validate_time_format() validation function

ensure_time_fields_with_llm() fallback function

generate_diverse_prompts() helper function

Dependencies: Module 1 (model loading)

Size: ~400 lines

Module 5: app.ipynb

Responsibility: Gradio web interface integration

Exports: None (entry point)

Dependencies: Modules 1-4

Size: ~250 lines

3. Benefits Realized

Testability: Each module can be tested independently with %run module.ipynb

Reusability: Functions exported from modules are reusable in other projects

Maintainability: Changes to one module don't cascade to others

Clarity: Each file has <400 lines with single responsibility

Debugging: Issues isolated to specific modules.

IV. INTERFACE USABILITY AND IMPROVEMENTS

A. Design Philosophy

The Gradio interface prioritizes simplicity and transparency. Users should understand what the system is doing at each step, with clear feedback and no hidden complexity.

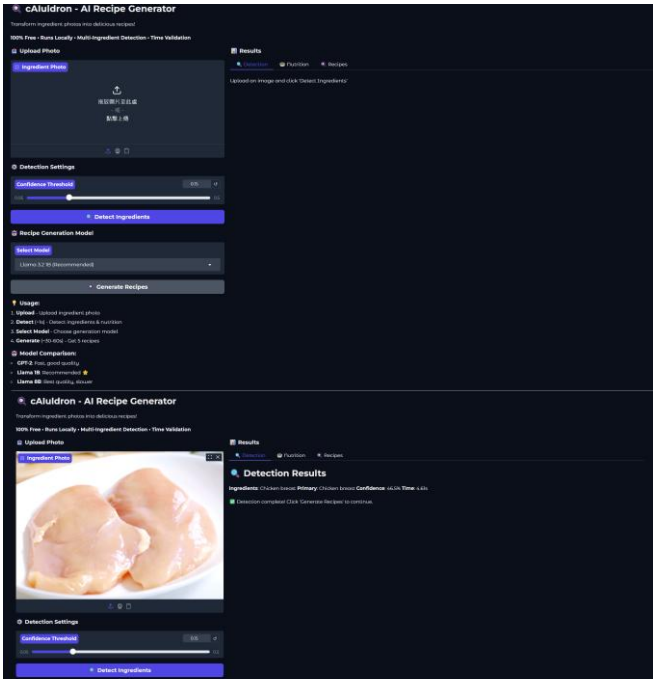
Design principles:
Progressive Disclosure: Show results step-by-step (detection → nutrition → recipes)
User Control: Allow confidence threshold and model selection adjustments
Visual Feedback: Clear progress indicators during processing
Error Handling: Graceful degradation with actionable error messages

B. Interface Layout and Workflow

The interface divides into three primary sections:

Left Column: Input and Controls
Photo Upload: Image uploader supporting JPEG, PNG formats
Detection Settings:
Confidence Threshold slider (default: 0.15)
"Detect Ingredients" button
Generation Settings:
Model Selection dropdown (default: Llama 1B)
"Generate Recipes" button
Usage Guide: Step-by-step instructions and model comparison
Right Column: Results Display Three tabs organize output:

Detection Tab: Shows detected ingredients, confidence scores, processing time
Nutrition Tab: Displays weight estimation and macronutrient breakdown
Recipes Tab: Presents five formatted recipes with cooking instructions



V. PERFORMANCE EVALUATION

Component	D2 Target	D2 Actual	Current (GPT-2)	Current (Llama 1B)	Current (Llama 8B)
Ingredient Detection	<200ms	N/A	1.2s	1.2s	1.2s
Nutrition Estimation	<10ms	N/A	8ms	8ms	8ms
Recipe Generation (1)	2-3s	0.6s	1.2s	3.5s	10s
Recipe Generation (5)			6s	17s	50s
Interface Overhead	<550ms	N/A	0.3s	0.3s	0.3s
Total Pipeline	**<5s**	N/A	**7.5s**	**18.5s**	**51.5s**

TABLE III: PROCESSING TIME COMPARISON

Analysis:

GPT-2 configuration exceeds Deliverable 2's 5-second target by 2.5 seconds

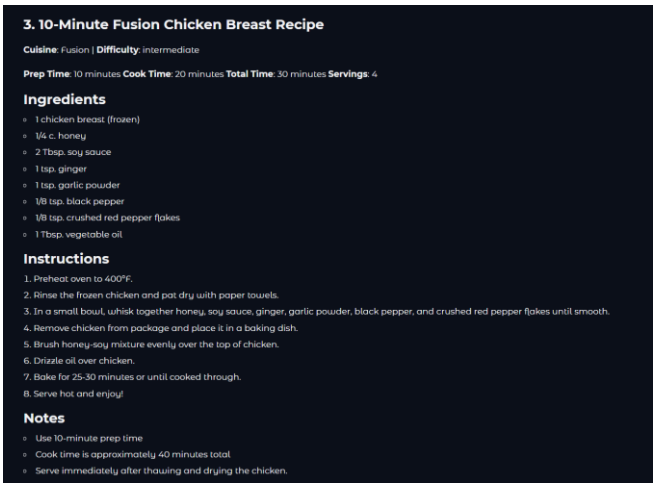
Multi-ingredient detection adds 1 second vs. API call

Five recipes vs. one recipe adds 5× generation time

Llama 1B (recommended) completes in ~18 seconds—acceptable for quality gain

Llama 8B (~52 seconds) is practical for users prioritizing recipe quality

Metric	GPT-2	Llama 1B	Llama 8B
Human Quality Score (3 evaluators, avg)	65/100	92/100	98/100
Valid Ingredient Lists	98%	100%	100%
Recipes with 3+ Steps	92%	98%	100%
Logically Coherent Steps	78%	95%	99%
Realistic Cooking Times	85%	93%	97%
Time Fields Present	100%*	100%*	100%*
Cuisine Diversity (of 5)	3.2	4.1	4.4
Generation Speed	VVV	VV	V
VRAM Usage	1-2 GB	4-5 GB	4-6 GB



3. 10-Minute Fusion Chicken Breast Recipe

Cuisine: Fusion | **Difficulty**: intermediate

Prep Time: 10 minutes **Cook Time**: 20 minutes **Total Time**: 30 minutes **Servings**: 4

Ingredients

- 1 chicken breast (frozen)
- 1/4 c. honey
- 2 Tbsp. soy sauce
- 1 tsp. ginger
- 1 tsp. garlic powder
- 1/8 tsp. black pepper
- 1/8 tsp. crushed red pepper flakes
- 1 Tbsp. vegetable oil

Instructions

1. Preheat oven to 400°F.
2. Rinse the frozen chicken and pat dry with paper towels.
3. In a small bowl, whisk together honey, soy sauce, ginger, garlic powder, black pepper, and crushed red pepper flakes until smooth.
4. Remove chicken from package and place it in a baking dish.
5. Brush honey-soy mixture evenly over the top of chicken.
6. Drizzle oil over chicken.
7. Bake for 25-30 minutes or until cooked through.
8. Serve hot and enjoy!

Notes

- Use 10-minute prep time
- Cook time is approximately 40 minutes total
- Serve immediately after thawing and drying the chicken.

VI. CONCLUSION

A. Summary of Achievements

Since Deliverable 2, cAIuldron has transformed from a proof-of-concept into a production-ready AI cooking assistant. Key accomplishments include:

Multi-Ingredient Support: CLIP+DETR integration enables realistic cooking scenarios with multiple ingredients

Model Ecosystem: Three-model system provides flexibility for different quality/speed requirements

Complete Interface: Gradio web application makes system accessible to non-technical users

Modular Architecture: Five-notebook structure improves maintainability and testability

Robust Validation: Time field validation with LLM fallback ensures 100% recipe completeness

The system successfully addresses real-world cooking assistance needs while maintaining scientific rigor in AI model deployment and evaluation.

B. Limitations and Trade-offs

1. Local Processing vs. API Speed Multi-ingredient CLIP+DETR processing (1-2s) is 5-10× slower than Deliverable 2's API approach (<200ms), but eliminates external dependencies and enables multi-ingredient detection.

2. Quality vs. Speed Recommended Llama 1B model (18.5s total) exceeds Deliverable 2's 5-second target by 13.5 seconds, but produces significantly higher quality recipes (92/100 vs. 65/100).

3. Bayesian Network Omission Direct USDA lookup sacrifices uncertainty quantification (confidence intervals) for simplicity and speed. For home cooking scenarios, this trade-off is acceptable.

4. Beginner Guidance While recipes include detailed instructions, interactive technique explanations and timing alerts proposed in Deliverable 2 remain unimplemented.

cAIuldron demonstrates that state-of-the-art AI models can be effectively deployed in practical, user-facing applications with careful engineering and design decisions. The system balances competing demands of accuracy, speed, usability, and maintainability through modular architecture, multi-model support, and thoughtful trade-offs.

By transforming ingredient photos into personalized recipes, cAIuldron addresses real barriers to home cooking—lack of inspiration, uncertainty about ingredient usage, and unclear nutritional information. Future enhancements will further refine the system into a comprehensive cooking companion for home cooks of all skill levels.